

KG-Agent 中文精读导读：让一个“小”模型自己在知识图谱上走多跳推理

原文：Jiang, Zhou, Zhao, Song, Zhu, Zhu, Wen (ACL 2025 / arXiv 2402.11163)

本导读为面向初学者的中文学习笔记：含要点翻译、方法精讲、框架图解读与关键结果重述。

论文卡片

标题	KG-Agent: An Efficient Autonomous Agent Framework for Complex Reasoning over Knowledge Graph
作者	Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, Yang Song, Chen Zhu, Hengshu Zhu, Ji-Rong Wen
机构	中国人民大学高瓴人工智能学院、信息学院；BOSS 直聘
发表	ACL 2025 (arXiv 预印本 2402.11163, 2024 年 2 月)
方向	知识图谱 (KG) × 大模型 × Agent × 指令微调
一句话	用约 1 万条“代码式”指令微调一个 7B 小模型，让它自己决定每一步调哪个工具、在知识图谱上多跳推理，超过用更大模型或更多数据的 SOTA 方法。

1 一、这篇论文在解决什么问题

大语言模型 (LLM, 如 GPT、LLaMA) 光靠“脑子里记住的知识”去回答**复杂的事实性问题**时会力不从心，尤其是**多跳 (multi-hop)**、**知识密集型**的问题。比如“C 罗在 2011 年效力的球队里，哪一支成立得最晚？”——这需要先查“C 罗 2011 年效力哪些队”，再查“每支队成立于哪一年”，最后“比大小取最晚的”，环环相扣。

知识图谱 (Knowledge Graph, KG) 恰好以“图”的方式存了海量事实，正好可以给 LLM 补充外部知识。但 KG 数据量大、又是结构化的，LLM 不容易直接用好它。已有做法主要分两类：

1. **检索增强 (retrieval-augmented)**: 先从 KG 里捞出相关三元组、拼成一段文字塞进提示里。缺点是丢掉了图的结构信息，还容易捞进一堆无关知识。
2. **协同增强 (synergy-augmented)**: 设计一套“LLM 与 KG 反复交互”的机制，让 LLM 多次查询 KG。效果更好，但作者指出它有两个硬伤：

- **交互流程是“写死的”**：往往按人工设计好的固定多轮计划走，遇到难度或约束不同的新题型就不灵活；
- **依赖闭源大模型**：大多要靠 ChatGPT / GPT-4 这类强力闭源 API 去理解或学习解题，而它们蒸馏出来的“计划”未必适合指导更弱的小模型。

本文的目标：做一个自主 (autonomous) 的、基于 LLM 的 Agent 框架——叫 **KG-Agent**——让一个相对“小”的 7B 模型能自己做决定，一步步在 KG 上推理直到找到答案，**全程不需要人类干预、也不依赖闭源 API**。

2 二、摘要中译

本文旨在提升大语言模型 (LLM) 在知识图谱 (KG) 上的推理能力，以回答复杂问题。受已有“设计 LLM 与 KG 交互策略”工作的启发，我们提出一个基于 LLM 的自主 Agent 框架，称为 KG-Agent，它使一个小型 LLM 能够**主动做决策**，直至完成在 KG 上的整个推理过程。在 KG-Agent 中，我们集成了 LLM、多功能工具箱 (multifunctional toolbox)、基于 KG 的执行器 (KG-based executor) 以及知识记忆 (knowledge memory)，并设计了一套**迭代机制**，它能自主地选择工具、随后更新记忆，从而在 KG 上推理。为保证效果，我们利用程序语言 (program language) 来表述 KG 上的多跳推理过程，并合成一个**基于代码的指令数据集**来微调基座 LLM。大量实验表明：仅用 **1 万 (10K) 条样本**微调 LLaMA-7B，就能在域内 (in-domain) 和域外 (out-of-domain) 数据集上**超过那些使用更大 LLM 或更多数据**的当前最优方法。我们的代码与数据将公开发布。

3 三、三大核心贡献

1. **为 KG 量身定做一个工具箱**：把“操作结构化数据”的能力外挂给 LLM，让它能在 KG 与中间结果上做**筛选、计数、检索**等离散或高级操作（共三类工具，详见第七节）。
2. **用现成数据“反向”合成代码式指令数据**：不去蒸馏 GPT-4，而是利用已有 KGQA 数据集里标注好的 SQL 查询，先把它“翻译”成 KG 推理程序（一串函数调用），再拆成多步的输入-输出对，做成指令微调数据。
3. **提出自主迭代机制**：基于“**选工具 + 更新记忆**”的循环，把微调后的 LLM、工具箱、KG 执行器、知识记忆四者串起来，实现对 KG 的自主推理。

4 四、预备知识（小白先看这里）

【小白补丁】什么是知识图谱（Knowledge Graph, KG）

KG 用“图”来存事实，基本单位是三元组（triple） $\langle e, r, e' \rangle$ ，意思是“头实体 e 与尾实体 e' 之间存在关系 r ”。例如 $\langle \text{C 罗}, \text{效力球队}, \text{葡萄牙国家队} \rangle$ 。很多三元组连起来就织成一张大图。形式化地： $G = \{\langle e, r, e' \rangle \mid e, e' \in E, r \in R\}$ ，其中 E 是实体集、 R 是关系集。每个实体还有一个类型 t （如“国家”“人物”）。常见的大型 KG 有 Freebase、Wikidata。

【小白补丁】什么是多跳推理（multi-hop reasoning）

答案离问题中提到的实体“隔了好几步”。比如 WebQSP 的答案一般在主题实体的 2 跳以内，CWQ 可达 4 跳。每“跳”一步就相当于在图上沿一条关系走一格。多跳问题往往还带约束（如“=2011 年”）或数值操作（如“取最晚”）。

【小白补丁】什么是 Agent（智能体）与工具调用（tool calling）

Agent 指让 LLM 不只“一次性吐答案”，而是像人一样反复“观察—决策—行动”：看一眼当前状态，决定下一步干什么，调用一个外部“工具”（一段能执行的函数/程序），拿到结果后再继续。代表性思路是 ReAct（把 LLM 变成会“边想边做”的语言智能体）。本文的工具就是一组“在 KG 上查询/筛选/计数”的函数。

【小白补丁】什么是指令微调（instruction tuning / SFT）

拿“输入 $\rightarrow$ 期望输出”这样的成对样本去继续训练一个预训练好的 LLM，让它学会按指令办事。本文的特别之处在于：输入是“问题 + 工具箱说明 + 当前已知的 KG 信息 + 历史程序”，输出是“下一步该调用的那个函数”——于是模型学会的不是“背答案”，而是“在每一步该调哪个工具”这种决策能力。训练时只在“输出 y ”上算交叉熵损失：

$$\mathcal{L} = - \sum_{k=1}^m \log \Pr(y_k \mid x, y_{<k}). \quad (1)$$

其中 x, y 是某一步的输入与标准答案， m 是输出 y 的 token 数， $y_k, y_{<k}$ 是第 k 个及之前的 token。

直白说：给定输入和已经写出的部分，让模型预测下一个 token 尽量贴近标准答案。

5 五、数据与任务：在哪些“考卷”上比

KG-Agent 在两类任务上评测：基于 KG 的问答 (KGQA, 域内) 和开放域问答 (ODQA, 域外)。

- **域内 (in-domain)**：用四个常用 KGQA 数据集做指令微调与测试——**WebQSP** (答案在 2 跳内)、**CWQ** (在 WebQSP 上加约束、更难, 可达 4 跳)、**GrailQA** (专门考“泛化”, 分 I.I.D./组合/零样本三档), 前三者基于 **Freebase**; 以及 **KQA Pro** (基于 **Wikidata**, 需要组合推理、多跳、数值比较、集合运算等多种能力)。
- **域外 (out-of-domain)**：微调后不再训练, 直接零样本测三个 ODQA 数据集——**WQ**、**NQ**、**TQ** (分别记作 WQ-Freebase、NQ-Wiki、TQ-Wiki, 即只保留能链接到 KG 的问题)。
- **换领域 KG**：还在 **MetaQA** (电影领域 KG, 含 1/2/3 跳三个子集) 上测“迁移到新 KG”的能力。

【小白补丁】两个常用评测指标

Hits@1：排在第一位的答案是否命中正确答案 (看“头名准不准”); **F1**：综合考虑“查得全不全 (召回)”和“查得准不准 (精确)”, 适合答案是一个集合的情况。两者都是越高越好。

关键设置：指令微调数据总共只采样 10,000 条, 按 **WebQSP : KQA Pro : GrailQA : CWQ = 1 : 5 : 5 : 10** 的比例从域内数据集里抽取; 基座模型用 **LLaMA2-7B**; 学习率 $2e-5$ 、batch size 256、最大长度 1500、训练 3 个 epoch。(推理过程默认假设问题里提到的实体已经给定。)

6 六、方法逐节精讲

KG-Agent 的核心是一个被“调教好”的 LLM, 它能在 KG 上推理时自主做决策。整套方法分三块: (4.1) 为 KG 设计工具箱; (4.2) 合成代码式指令数据并微调小模型; (4.3) 设计自主推理框架。总体框架见图 1。

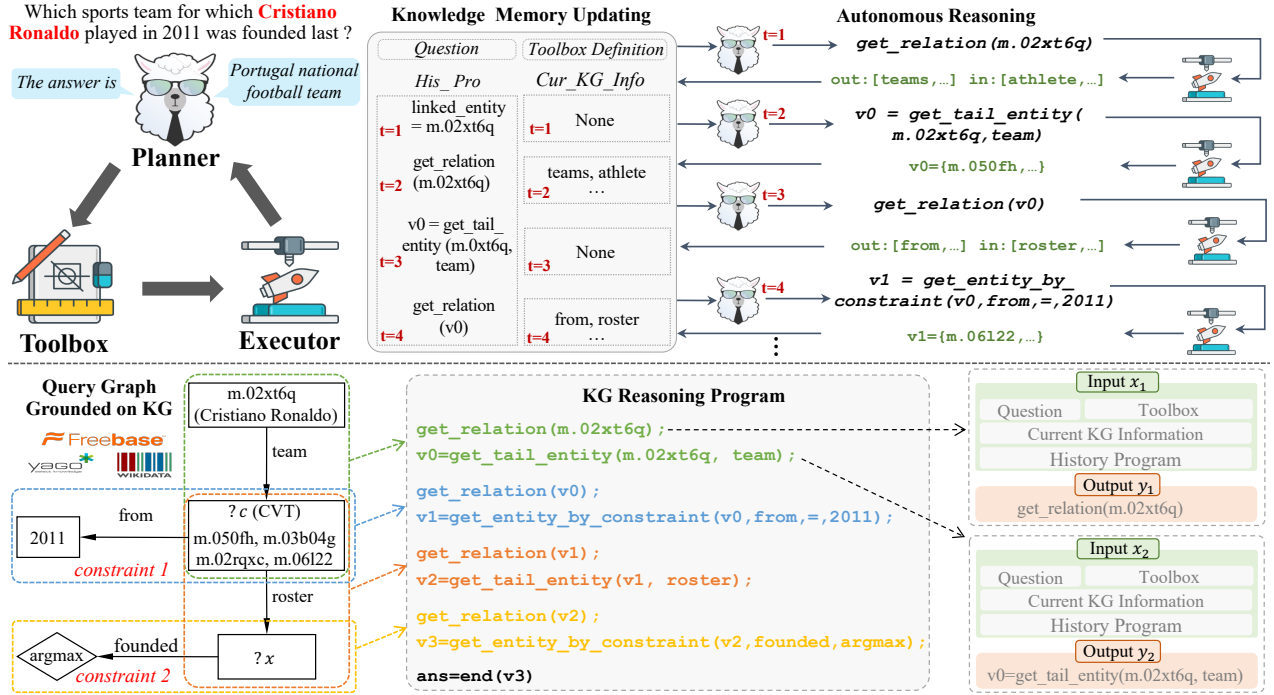


图 1: KG-Agent 总体框架 (原文 Figure 1, 图内英文为原文保留)。上半部分是 Agent 的工作流: 以问题 “C 罗 2011 年效力的球队里哪支成立最晚?” 为例, Planner (规划器, 即微调后的 LLM) 根据知识记忆在每一步选一个工具, Executor (执行器) 用程序编译器去 KG 上真正执行该工具调用, 并把新得到的实体/关系写回知识记忆 (中间 “Knowledge Memory Updating” 一栏展示了 $t = 1, 2, 3, \dots$ 各步记忆的变化), 如此循环直到调用 end 结束 (右侧 “Autonomous Reasoning”)。下半部分展示了指令数据的合成: 把标注 SQL 接地 (grounding) 到 KG 得到查询图 (Query Graph), 再转成一串 KG 推理程序 (get_relation、get_tail_entity、get_entity_by_constraint...end), 并按统一模板拆成多个 “输入 x_t / 输出 y_t ” 对。

6.1 4.1 为知识图谱设计工具箱

LLM 不擅长精确操作结构化数据, 于是作者为它配一套 “工具”。按 “在 KG 上推理通常需要的三种基本操作” —— 从 KG 取信息、按语义筛掉无关信息、对取出的信息做运算 —— 设计了三类共若干工具:

- **抽取类 (Extraction):** 访问 KG 本身。如取关系 get_relation、取头/尾实体 get_head_entity/get_tail_entity、按类型取实体 get_entity_by_type、按约束取实体 get_entity_by_constraint (支持 $=, >, \geq, <, \leq$ 比较, 或 argmax/argmin 取极值)。
- **逻辑类 (Logic):** 对取出的实体集做运算。如计数 count、求交 intersect、求并 union、条件判断 judge, 以及结束推理并返回最终答案的 end。
- **语义类 (Semantic):** 借助预训练模型实现的高级功能。如关系检索 retrieve_relation (从候选关系里挑出与问题语义最相关的) 和实体消歧 disambiguate_entity。

【小白补丁】为什么这样设计很灵活

每个工具的“输入 → 输出”格式都按统一方式定义好了（例如多数工具的输入/输出都是“实体集合 {e}”），所以工具箱可以**按真实需求随时扩展**，框架不挑具体 KG，也不挑任务类型。

6.2 4.2 合成代码式指令数据并微调小模型

思路：不蒸馏 GPT-4，而是“废物利用”现成 KGQA 数据集里标注好的 SQL 查询。SQL 里本来就含有关系链、条件、约束等信息，正好拿来合成“推理程序”。流程分三步：

(1) **推理链抽取：**KG 太大，先用规则匹配从 KG 上拿到一小块与问题相关的**查询图 (query graph)**——它是树状的、能直接对应到逻辑形式，清晰刻画了“为得到答案，SQL 是怎么一步步执行的”。再从问题里提到的实体（如“C 罗”）出发，用**广度优先搜索 (BFS)** 遍历查询图所有节点，得到一条把起点实体连到答案实体的**推理链**（如 `teams → roster_team`），途中自然带上约束（如 `roster_from = “2011”`）和数值操作（如 `founded` 取“最晚”）。

(2) **推理程序生成：**把推理链转成若干**相互关联的三元组**（每个三元组 一个中间推理步），再把三元组改写成**代码格式的函数调用**（代表“调用某个工具”，且能真去 KG 上执行拿到对应三元组）。具体地，从 `get_relation(e)` 拿到当前候选关系，选一个关系传给 `get_tail_entity` 等函数得到新实体，按推理链顺序生成所有函数调用，拼成完整的 **KG 推理程序**（见图 1 下半部分的代码示例）。

(3) **指令合成（输入-输出对构造）：**把这段程序里**每一个函数调用**都做成一条指令——**输入** = 问题 + 工具箱定义 + 当前 KG 信息（即当前实体集的下一批候选关系）+ 此步之前的历史程序；**输出** = 当前这一步的函数调用。执行完当前步后，“历史程序”和“当前 KG 信息”会相应更新，输出顺延为下一步的函数调用。这样每条原始样本就被拆成一串“**输入-输出对**”，完整刻画了在 KG 上的推理轨迹。对每条样本，把整条轨迹写成 $\{\langle x_1, y_1 \rangle, \dots, \langle x_n, y_n \rangle\}$ (n 为总步数)，喂给**仅解码器 (decoder-only)** 的 LLaMA-7B 做 SFT，损失见公式 (1)。

6.3 4.3 自主推理：四个组件 + “选工具/更记忆”循环

微调完之后，作者把它装进一个 Agent 框架（图 1 上半部分），含**四个组件**：

1. **LLM 规划器 (Planner)：**就是 4.2 微调出的那个 LLM，负责“看记忆、选工具”。
2. **多功能工具箱 (Toolbox)：**4.1 里那套工具。
3. **KG 执行器 (Executor)：**用程序编译器真正执行函数调用，能缓存/操作中间变量、从 KG 抽出新实体或关系。
4. **知识记忆 (Knowledge Memory)：**记录全程的上下文与“当前有用信息”。

知识记忆的四部分： 自然语言问题、 工具箱定义、 当前 KG 信息、 历史推理程序。前两部分用给定的问题和工具箱初始化，**全程不变**；后两部分初始化为空列表，**每一步都更新**。

整个推理是一个“**选工具 → 执行 → 更新记忆**”的循环：

- **规划器选工具：** 把当前记忆按模板拼成输入，LLM 从中**生成一个函数调用**（选一个工具 + 它的参数）。它要应对四类需求：把提到的实体链接到 KG、访问 KG 信息、处理中间结果、或调用 end 返回最终答案。
- **执行器更记忆：** 执行该函数调用后更新记忆——把这次调用追加进“历史程序”；若该工具是去 KG 取新信息（如 `get_relation`），就把结果加进“当前 KG 信息”。
- **自主迭代：** 不断重复上面两步做逐步推理。作者打了个比方：这个多轮决策过程就像“**沿着关系在 KG 上行走**”。一旦走到答案实体，Agent 会**自动停止**。整个过程**不依赖具体任务类型或具体 KG**，因此是个通用框架。

【小白补丁】和已有工作的关键区别（对照原文 Table 1）

原文用一张表对比了 Pangu、StructGPT、RoG、ChatDB、KB-BINDER 等方法。KG-Agent 的与众不同在于同时满足：交互方式是**自主（auto）**而非写死（pd）、基座是**开源 LLaMA2-7B**（不靠闭源 API）、**同时具备工具（Tool）与记忆（Memory）**、并支持**跨多个 KG 的多任务（Multi-Task）泛化**——这一整套组合是它独有的。

7 七、实验与结果：1 万样本的 7B 如何超过 SOTA

对比对象： 域内比三类基线——子图推理（如 NSM、UniKGQA）、LM 序列生成（如 RNG-KBQA、TIARA、PanGu）、LLM 方法（如 RoG、StructGPT，以及 ChatGPT/Davinci-003/GPT-4）；域外比微调类（T5/BART）和 LLM 类。

域内主结果（基于 Freebase 的 WebQSP/CWQ/GrailQA，数值源自原文 Table 2，节选关键行；越高越好，加粗为最优）：

表 1: 基于 Freebase 的域内主结果对比（数值源自原文 Table 2）。GrailQA 报告 F1。

方法	WebQSP		CWQ		GrailQA (F1)	
	Hits@1	F1	Hits@1	F1	Overall	Zero-shot
UniKGQA（子图推理）	75.1	70.2	50.7	48.0	—	—
RoG（LLM 方法）	85.7	70.8	62.6	56.2	—	—
FC-KBQA（LM 生成）	—	76.9	—	56.4	83.8	83.1
PanGu w/ T5-3B	—	79.6	—	—	83.4	—
GPT-4	73.2	62.3	55.6	49.9	31.7	39.2
StructGPT	72.6	63.7	54.3	49.6	54.6	50.5
KG-Agent（本文）	83.3	81.0	72.2	69.8	86.1	86.3

怎么读这张表：KG-Agent 在 F1 上几乎全面领先——WebQSP 的 F1 达 81.0（高于 PanGu-3B 的 79.6），CWQ 的 F1 达 69.8、GrailQA 的 Overall F1 达 86.1，都明显超过此前最优。直接用 GPT-4/ChatGPT 与最好的微调方法仍有较大差距，说明“光靠 LLM 硬答复杂问题”很难。

【小白补丁】关键数字：少数据、小模型，却更强

只用约 10K 条样本微调一个 7B 小模型，就超过了那些用更大 LLM 或全量数据的方法。原文给出更直观的对比：相比基线，KG-Agent 在 CWQ 和 GrailQA 上分别只用了约 36% 和 23% 的原始训练集量，却分别取得 7.5% 和 2.7% 的 F1 相对提升；在混合数据上联合训练（多数据集相互增益）后，WebQSP / CWQ / GrailQA 的 F1 又分别再涨 1.7% / 7.5% / 2.7%。

基于 Wikidata 的 KQA Pro（数值源自原文 Table 3，准确率，越高越好）：

表 2: KQA Pro 测试集准确率（源自原文 Table 3，节选）。Multi-hop/Comparison/Count 等为按推理类型细分的子集。

方法	Overall	Multi-hop	Comparison	Logical	Count	Zero-shot
BART SPARQL	89.68	88.49	96.12	88.67	85.78	87.88
GPT-4	37.43	34.82	55.75	36.81	15.27	27.28
ChatGPT	24.96	24.22	39.15	25.51	10.76	15.67
KG-Agent（本文）	92.15	91.03	96.32	91.28	88.21	91.40

KG-Agent 在 KQA Pro 总体准确率达 92.15，全面超过强力的 BART SPARQL (89.68)；而 GPT-4/ChatGPT 在这类需要组合与数值推理的题上表现很差（如 Count 子集仅 15.27 / 10.76），凸显“配上工具与 KG”的价值。

域外零样本（源自原文 Table 4）：微调后不再训练，直接零样本测 ODQA。在基于 Freebase 的 WQ-Freebase 上，T5/BART 即使全量微调也答不好；KG-Agent 因为“只需学会与 KG 交互、不必死记具体知识”，零样本就比这些全量监督模型更好（原文摘要：WQ-Freebase 与 TQ-Wiki 上准确率分别相对提升约 9.7% 和 8.5%）。

迁移到领域 KG（源自原文 Table 5，MetaQA 电影 KG）：在 1/2/3 跳子集上，KG-Agent (97.1 / 98.0 / 92.1) 一致超过 StructGPT 与监督微调基线，说明它学到的是“**在 KG 上推理的通用能力**”，能高效迁移到新 KG。

进一步分析：

- **指令数量的影响（原文 Figure 2）：**把总量从 2k 指数级加到 64k，性能随数据增多而上升、最终趋于稳定；从 16k 到 64k 提升已不明显——作者认为这与数据“多样性”有关，未来会构造更多样的样本。这也佐证了“**约 10K 已足够**”的结论。
- **采样比例的影响（原文 Table 6）：**在总量不变下，把某个数据集的比例翻倍，它自己的分数会涨，但三者平均分都不如作者选定的 1:5:5:10 比例——说明该比例较好地平衡了综合能力。

8 八、小白知识点卡片（速查）

- **知识图谱 / KG：**用三元组 $\langle e, r, e' \rangle$ 存事实的图（Freebase、Wikidata 等）。
- **多跳推理：**答案离问题实体隔多步，常带约束与数值操作。
- **Agent / 工具调用：**让 LLM “观察—决策—行动”循环，调用可执行的外部函数。
- **指令微调 (SFT)：**用“输入 → 输出”成对样本继续训练 LLM；本文教模型学“每步该调哪个工具”。
- **KGQA / ODQA：**基于 KG 的问答 / 开放域问答（本文域内 / 域外任务）。
- **查询图 (query graph)：**从 SQL 接地到 KG 得到的树状小图，刻画求解的执行流。
- **知识记忆：**问题 + 工具箱定义 + 当前 KG 信息 + 历史程序，后两者每步更新。
- **Hits@1 / F1：**头名是否命中 / 综合查准查全；越高越好。
- **SPARQL / SQL：**在图数据库 / 关系库上查询的结构化语言（本文借标注 SQL 反向合成程序）。

9 九、一句话总结与谱系定位

一句话：KG-Agent 把“在 KG 上做多跳推理”改写成一段可执行的代码式轨迹——先用现成 KGQA 数据集的标注 SQL 反向合成“代码式指令数据”，用约 1 万条就把一个 7B 小模型调成“规

划器”；再让它在“**选工具** → **执行** → **更新知识记忆**”的循环里自己决定**每一步**，像“沿关系在图上行走”一样找到答案，从而**又准、又省、又不依赖闭源大模型**地超过更大/更多数据的 SOTA。

在“**LLM × 知识图谱推理**”谱系里的位置：它属于“**协同增强 + 自主 Agent + 小模型指令微调**”这一路线的代表作。与“写死交互流程”的 StructGPT/KB-BINDER/RoG 相比，它的**交互流程是模型自主决定的**；与依赖 GPT-4 的 ChatDB 等相比，它**只靠一个开源 7B 并额外配了工具去做 count 等专门操作**；与“先生成完整计划再接地”的方法相比，它**边走边决策**、灵活适配不同难度与约束。你可以把它和本系列里走“图检索 + 软提示”的 G-Retriever、走“图上思维链”的 GraphCoT、走“强化学习”的方法**对照着看**——它们各自把“图的信息”用进 LLM 的方式很不一样。

动手提示：复现的关键三步是——（1）从标注 SQL 合成 KG 推理程序与输入-输出对；（2）用 1:5:5:10 比例约 10K 样本微调 LLaMA2-7B；（3）实现“规划器 + 工具箱 + KG 执行器 + 知识记忆”的迭代循环。论文称代码与数据将公开发布。